

Capítulo 2: El Paradigma Híbrido: POO Pura y Programación Funcional (PF)

1. Explicación Teórica: La Fusión de Paradigmas

Scala (acrónimo de *Scalable Language*) fue diseñado por Martin Odersky con el propósito central de **fusionar la Programación Orientada a Objetos (POO) y la Programación Funcional (PF) en una sola ontología**. Este enfoque multiparadigma permite a los desarrolladores aprovechar la madurez de la POO en la JVM y la expresividad y seguridad de la PF.

POO Pura (Todo es un Objeto)

A diferencia de Java o C++, que distinguen entre tipos primitivos y objetos de referencia, Scala adopta un modelo de objetos **puro: todo valor es un objeto y toda operación es una llamada a un método**.

- **Jerarquía Unificada:** Todos los tipos en Scala descienden de un tipo raíz común llamado `Any`. Los tipos de valor (`Int`, `Boolean`), conocidos como `AnyVal`, y los tipos de referencia (`AnyRef`, equivalentes a `java.lang.Object`), están unidos bajo esta misma jerarquía. Esta uniformidad permite la escritura de código genérico que funciona consistentemente con números y objetos complejos.

Programación Funcional (Funciones de Primera Clase)

Scala incorpora la PF al tratar las **funciones como valores de primera clase**. Esto significa que una función es tratada como cualquier otro dato (como un entero o una cadena de texto).

- **Composición Funcional:** Las funciones pueden ser **asignadas a variables**, pasadas como argumentos a otras funciones (Funciones de Orden Superior o HOFs), o devueltas como resultado. HOFs como `map`, `filter`, y `reduce` son esenciales para transformar colecciones de datos de manera declarativa, sin recurrir a bucles imperativos.
- **La Clave Arquitectónica:** Internamente, una función de Scala (`FunctionX`) es compilada a un objeto, una instancia de un *trait* con un método `apply`. Esta es la razón técnica por la que la JVM, diseñada para objetos, puede ejecutar sin problemas el paradigma funcional.

El Sello de Inmutabilidad: `val` vs. `var`

La inmutabilidad no es una mera preferencia estilística, sino una **decisión de ingeniería fundamental** en Scala para construir **sistemas concurrentes seguros**.

Palabra Clave	Propósito	Mutabilidad	Uso Idiomático
<code>val</code>	Valor (Value). Es una referencia inmutable. Su valor no puede cambiar una vez asignado.	Inmutable	Usar por defecto. Equivale a <code>final</code> en Java.
<code>var</code>	Variable (Variable). Es una referencia mutable. Su valor puede ser reasignado.	Mutable	Evitar , se limita a casos de optimización o manejo de estado controlado (ej. dentro de un Actor).

Programación Orientada a Expresiones (*Expression-Oriented Programming*)

En Scala, **casi todas las estructuras de control se evalúan y devuelven un valor** (son expresiones).

- **Retorno Implícito:** El valor de una función o bloque de código (`{ }`) es el valor de la **última expresión evaluada**. Esto hace que la palabra clave `return` sea innecesaria en la mayoría de los casos.

2. Sintaxis y Ejemplos de Código (Scala 3)

Los siguientes ejemplos ilustran la inmutabilidad por defecto y el uso de estructuras de control como expresiones, pilares de la programación idiomática en Scala 3.

Inferencia de Tipos e Inmutabilidad

Scala es fuertemente tipado estáticamente, pero el compilador es **lo suficientemente potente para inferir el tipo** basándose en el valor asignado.

```

1  // El compilador infiere Int (entero)
2  val numeroMagico = 42
3  // numeroMagico: Int = 42
4
5  // Intentar reasignar un val causa un error de compilación
6  // numeroMagico = 99
7  // Error: reassignment to val
8
9  // Uso de var (mutable)
10 var contador: Int = 0
11 // El uso de var está desaconsejado, pero es necesario para la mutabilidad
    explícita.
```

```
12 contador = contador + 1 // Válido
```

Control de Flujo como Expresiones

La estructura `if-else` y los bloques de código devuelven un valor.

```
1 // Definición de una función simple
2 // El compilador infiere que retorna String, eliminando la necesidad de
  escribir ': String'
3 def clasificarEdad(edad: Int) =
4   // If-Else como expresión (no se necesita el operador ternario de
    C/Java).
5   val tipo =
6     if edad >= 65 then
7       "Jubilado"
8     else if edad >= 18 then
9       "Adulto"
10    else
11      "Menor"
12
13   // La última expresión del bloque (tipo) es retornada implícitamente.
14   tipo
15
16 // Bloque de código como expresión
17 val resultadoBloque: Int = {
18   val a = 5 // Variable local dentro del bloque
19   val b = 7
20   a + b // El valor del bloque es 12
21 }
```

3. Comparativa (Java y Python)

El paradigma híbrido de Scala resuelve problemas de verbosidad y seguridad inherentes a la POO tradicional y a los lenguajes dinámicos.

Concepto	Java (Imperativo/POO)	Python (Dinámico)	Scala (Híbrido/Funcional)
Inmutabilidad	Opcional, requiere <code>final</code> . Mutable por defecto.	Por convención; las variables son reasignables.	<code>val</code> (inmutable) es el valor por defecto. Mutabilidad explícita con <code>var</code> .
Funciones	Métodos atados a clases (<code>static</code> o instancia).	Valores de primera clase (lambdas).	Valores de primera clase encapsulados en

Concepto	Java (Imperativo/POO)	Python (Dinámico)	Scala (Híbrido/Funcional)
			objetos (<code>FunctionX</code> traits).
Jerarquía de Tipos	Distinción entre primitivos (<code>int</code>) y objetos (<code>Integer</code>).	Tipado dinámico.	Todo es un objeto (<code>Any</code>), eliminando la necesidad de <i>autoboxing</i> .
Control de Flujo	Estructuras como <code>if</code> son instrucciones (<i>statements</i>). Necesita <code>return</code> para devolver valor de un método.	Instrucciones o expresiones especializadas.	Todo es una expresión . El valor de retorno es implícito.

4. Best Practices (*The Scala Way*)

La forma idiomática de programar en Scala se centra en maximizar la inmutabilidad y la expresividad compositiva que ofrece la PF:

1. **Imponer la Inmutabilidad:** Declarar siempre `val` a menos que la mutabilidad sea estrictamente necesaria. Esto minimiza la posibilidad de *race conditions* y simplifica la lógica en sistemas concurrentes.
2. **Abrazar las Expresiones:** Evitar el uso explícito de `return` y modelar la lógica de negocios utilizando estructuras de control que retornan valor (`if-else` , `match`), asignando directamente el resultado a un `val` .
3. **Composición Funcional sobre Iteración:** Para trabajar con colecciones (`List` , `Vector` , `Set`), utilizar siempre **Funciones de Orden Superior (HOFs)** como `map` , `filter` , y `fold` en lugar de bucles `for` o `while` al estilo imperativo.
4. **Diseño Modular:** Utilizar `object` (Singleton) para funciones utilitarias y métodos que habrían sido `static` en Java, manteniendo la semántica de la POO pura.
5. **Seguridad de Tipos sin Redundancia:** Confiar en la **inferencia de tipos** del compilador para un código conciso. Solo declarar el tipo explícitamente cuando se define una API pública o cuando la inferencia podría resultar ambigua.